KV Cache – 1

Inference basics, KV cache memory math, and the move from Multi-Head to Multi-Query Attention

Item	Details
Class subject	KV cache – 1 (inference engineering)
Date	3 October 2026 (taken from the recording file name, GMT; please correct if different)
Tutor	Paul (as labelled in the transcript)
Length of recording	about 3 h 25 min
Notes level / style / language	Beginner | Detailed | English

Materials covered

File	What it is
GMT20261003-143207_Recording_..._transcript.vtt	Class transcript (timestamps shown as [hh:mm:ss])
KVCACHE.pdf	Tutor's handwritten whiteboard notes, 12 pages (cited as WB p.N)
1911_02150v1.pdf	Shazeer (2019), "Fast Transformer Decoding: One Write-Head is All You Need" (cited as Paper §N)
00_foundations.ipynb	Session 0: tokens, embeddings, self-attention, causal mask (NB0)
01_naive_decoding.ipynb	Session 1: cost of not caching (NB1)
02_kv_cache_memory_math.ipynb	Session 2: KV cache memory math (NB2)
03_mha_recap.ipynb	Session 3: MHA recap, prefill vs decode (NB3)
04_mqa.ipynb	Session 4: Multi-Query Attention (NB4)
Copy_of_KVCACHE_DEMO.ipynb	Colab demo: generation with vs without cache (Demo)

How to read these notes: notes are organised by topic in the order a beginner should study them, not in the order the tutor spoke. Coloured boxes mark Key Concepts, Examples, Tips, Warnings, Q&A, and anything that appears only in the class or only in the materials.

Contents

Contents 2

Topic Map: study order vs. tutor order 3

1. Foundations recap: tokens, QKV, causal mask, generation loop 4

2. Autoregressive property and the naive decoding problem 6

3. The KV cache idea: what it is, where it lives, history 8

4. LLM inference: prefill and decode, TTFT and TPOT 10

5. The memory wall: compute throughput vs memory bandwidth 12

6. KV cache memory math 14

7. KV cache in practice: lifetime, eviction, prompt cache 17

8. Multi-Head Attention (MHA) recap 19

9. Multi-Query Attention (MQA) 21

10. Demo: generation with and without the KV cache 24

11. The MQA paper: "Fast Transformer Decoding: One Write-Head is All You Need" . . . 25

12. Live Q&A; at the end of class [02:58]–[03:24] 26

13. Logistics, roadmap and tangents 27

Key takeaways 29

Glossary 29

Action items, homework and deadlines 30

Quick revision cheat sheet 32

Topic Map: study order vs. tutor order

The tutor jumped around (for example, he returned to "who needs the KV cache" in the final Q&A, and talked about cache clearing in four separate places). This map merges every appearance of each topic and gives the order a learner should study them in (prerequisites first).

Study #	Topic	Where it appears in the transcript	Tutor order	Materials
1	Foundations recap: tokens, QKV, causal mask, autoregressive loop	~[00:13]–[00:31]	1st	NB0; WB p.1
2	Autoregressive property and naive decoding (the problem)	[00:19]–[00:31], [00:31]–[00:47]	2nd	WB p.1; NB1
3	The KV cache idea (what, where, history)	[00:42]–[00:47], [01:48], [03:03]–[03:12]	3rd (revisited later)	NB1, NB2; WB p.1
4	LLM inference: prefill, decode, TTFT, TPOT	[00:47]–[01:04]	4th	WB p.2–3; NB3
5	The memory wall: compute vs bandwidth	[01:04]–[01:24]	5th	WB p.4–6
6	KV cache memory math	[01:36]–[01:55]	7th	WB p.6–7; NB2
7	KV cache in practice: batch, eviction, prompt cache	[01:48], [01:56], [02:28]–[02:38], [03:03]–[03:13], [03:21]–[03:23]	scattered	NB2 (batch only)
8	Multi-Head Attention (MHA) recap	[01:55]–[02:03]	8th	WB p.8; NB3
9	Multi-Query Attention (MQA)	[02:03]–[02:27], [02:54]–[02:58]	9th	WB p.9–12; NB4; Paper
10	Demo: generation with vs without cache	[02:27]–[02:44]	10th	Demo
11	The MQA paper (Shazeer 2019)	[02:45]–[02:53]	11th	Paper
12	Live Q&A at the end of class	[02:58]–[03:24]	12th	-
13	Logistics, roadmap, tangents	[00:00]–[00:13], [01:25]–[01:36], [03:24]	scattered	-

Tutor order in one line: admin → recap → naive decoding → prefill/decode → memory wall → break → course-planning chat → KV math → MHA → MQA → demo → paper → Q&A.

1. Foundations recap: tokens, QKV, causal mask, generation loop

Everything in KV caching is built on how a decoder turns text into tokens and generates one token at a time. The tutor said that without this, KV cache "doesn't make sense".

Key concepts

KEY CONCEPT

Simple: a model never sees words. Text becomes integer IDs (tokens); each ID is looked up in a table that gives a vector (the embedding).

Precise: sentence → tokens → token IDs → embeddings (vectorisation). In the notebook, *embedding_table* has shape (vocab_size, d_model); for "the cat sat" the IDs are [0, 1, 2] and the embeddings have shape (3, 8). The IDs in the notebook are not GPT/BERT IDs: no BPE, WordPiece or SentencePiece is used, to keep it simple.

KEY CONCEPT

Q, K, V. Every token produces a Query ("what am I looking for?"), a Key ("what do I offer, as a label?") and a Value ("what do I contribute if picked?") (NB0).

Scaled dot-product attention: Attention(Q,K,V) = softmax($Q \cdot K^T / \sqrt{d_k}$) · V. The $\sqrt{d_k}$ division stops dot products growing too large with dimension, which would make softmax too peaked (NB0). The tutor asked students to remember the formula "by heart".

Explanation

- **Dimensions.** The original "Attention Is All You Need" model used $d_{model} = 512$. The notebook starts smaller ($d_{model} = 8$, head dimension $d_k = 4$) so every number can be printed; later notebooks use $d_{model} = 32$.
- **Causal masking.** Before softmax, scores for future positions are set to $-\infty$, because softmax($-\infty$) = 0. This happens on the **decoder side** only (masked multi-head attention; also called causal language modelling, CLM). Each query can look at itself and the past, never the future.
- **Padding.** For decoders the tutor said padding is always **left padding** [00:17:35] (recalled from last week's class).
- **Autoregressive generation.** Run the model, take the last position's output, turn it into a token, append it, repeat. Because of the mask this is a sequential process: one token per step [00:17:53]. Generation stops at the max-new-tokens limit you set (e.g. 100, 200, 300). Top-k / top-p are sampling methods, not the stopping rule.
- **Where it lives in a model.** A decoder layer is self-attention → add & norm → feed-forward → add & norm (NB0). The whole course lives inside the self-attention block; the feed-forward part never changes between variants.

Example: who may attend to whom (tutor's walk-through)

Query position	Key position	Allowed?	Why
0	0	Yes	Same position
0	4	No (blocked)	Key 4 is in the future
4	0	Yes	Past
4	4	Yes	Itself

Causal mask for 5 positions from NB0 (X = blocked, . = allowed):

```
query\key   0  1  2  3  4
   0        .  X  X  X  X     -> row 0 weights: [1, 0, 0, 0, 0]  (only sees itself)
   1        .  .  X  X  X
   2        .  .  .  X  X
   3        .  .  .  .  X
   4        .  .  .  .  .     -> row 4 weights: [0.079, 0.218, 0.186, 0.234, 0.283]
```

Saved output of NB0, section 4 (random scores, seed 0).

MATERIAL REFERENCE

NB0 sections 1–6 and its two Mermaid diagrams; WB p.1 (agenda: "KV Cache; 1) Autoregressive Property (Dec)").

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB0 includes the library analogies: embedding table = "a huge dictionary where every word is replaced by coordinates on a map"; attention = a library search where the query is your search box, keys are subject tags, values are book contents.
- NB0 attention-by-hand output: with 3 tokens and head_dim 4, every row of attention weights sums to 1.
- NB0 toy generation loop (vocab 50, d_model 16, identity "transformer"): prompt of 3 tokens, 4 generated tokens: 25, 33, 9, 6.

SAID IN CLASS, NOT IN MATERIALS

- Left padding is the rule for decoders.
- Rough speed: modern LLMs take about 20–30 ms per generated token; with smaller models 30–40 ms (a rough estimate, "not for every model") [00:19:09].
- The mask is applied on top of the scores using a separate mask matrix; normal (bidirectional) attention and causal attention are different.

Q&A

- **Q:** Why not replace the start token with a padding token? **A:** Padding would add information; the model needs to be shown "this is the start, this is the end".
- **Q:** Is a CLS token used? **A:** CLS belongs to BERT-style encoders. The tutor used <*bos*> as a generic start token.
- **Q:** Can key position 4 access query position 0, i.e. do we access past queries? **A:** Anything in the past can be accessed, but generally a query looks at other tokens' K and V, not at other queries. Once a query has been processed, its K and V are saved.
- **Q:** Why do we need the "tiny causal attention" module? **A:** To demonstrate the autoregressive property: generation one token at a time.
- **Q:** Is stopping decided by top-k? **A:** No. Top-k/top-p are stochastic sampling; stopping is by the max-token setting.

2. Autoregressive property and the naive decoding problem

Without any cache, the model recomputes K and V for every old token at every step. That is wasted work, and it grows quadratically.

Key concepts

KEY CONCEPT

Autoregressive property: the model generates exactly one token at a time, and each new token needs attention over all earlier tokens.

Naive decoding: decoding with no cache. At decode step t (generating token t+1) you recompute K and V for all t previous tokens from scratch, even though tokens 1..t−1 never change (NB1).

Decoding in the tutor's usage always means "generating one token".

Explanation: the whiteboard walk-through

The tutor wrote the sentence "<bos> The capital of France is [<eos>]" and treated <eos> as the token still to be predicted (WB p.1). At each time step he listed which Ks and Vs get computed:

Step	Input so far	K computed	V computed
t1	<bos>	K<bos>	V<bos>
t2	<bos> The	K<bos>, K_{The}	V<bos>, V_{The}
t3	<bos> The capital	K<bos>, K_{The}, $K_{capital}$	V<bos>, V_{The}, $V_{capital}$
t4	<bos> The capital of	K<bos>, K_{The}, $K_{capital}$, K_{of}	V<bos>, V_{The}, $V_{capital}$, V_{of}
tn	<bos> The capital of France is	K<bos>, K_{The}, $K_{capital}$, K_{of}, K_{France}, K_{is}	V<bos>, …, V_{France}, V_{is}

Board note: "Every token = {K, V}" and "KV :- MHA", meaning the K and V calculation happens inside the multi-head attention block (under the hood, scaled dot-product attention) [00:30:37].

- **Even <bos> has a K and a V** (the tutor stressed this when students were unsure).
- **The repeated work.** K<bos> is computed at step 1 and then again at steps 2, 3, 4, … Only one new token's K and V is genuinely new at each step. With a 1,000-token sequence the first token's K/V would be recomputed about 999 times, the second about 998 times, and so on (tutor's example).
- **Not a big deal for 10 tokens**, but it becomes a problem as the context window grows.
- **Q is also needed** at each step (for the new token), but the discussion focuses on K and V because those are what get repeated.

Hand-trace and the numbers (NB1)

Decode step	Sequence length fed to model	Tokens whose K/V are recomputed
1	4	4 (all of them)
2	5	5 (all of them, again)
3	6	6
4	7	7
5	8	8

```
step 1: forward over tokens [1..4]      recompute K,V for 1,2,3,4
step 2: forward over tokens [1..5]      recompute K,V for 1,2,3,4,5      (1-4 redone!)
step 3: forward over tokens [1..6]      recompute K,V for 1,2,3,4,5,6    (1-5 redone!)
```

Text version of the NB1 sequence diagram.

The quadratic blow-up. Total attention work across a generation of n tokens with no cache is 1 + 2 + ... + n = n(n+1)/2. The tutor called this "the quadratic part" and said the aim is "not to blow up the number of calculations".

Total tokens generated n	Time (s) measured in NB1	n(n+1)/2 (relative work units)
8	0.0006	36
32	0.0017	528
128	0.0113	8,256
256	0.0262	32,896

In class the tutor rounded 128 tokens to "about 8,000" units. The times in the table are from a tiny toy model run once on the notebook author's machine.

TIP

If a student asks "why quadratic?": at each step you redo all earlier work, so the amounts 1, 2, 3, ... n add up to n(n+1)/2 ("simple maths", Divankar's question).

WARNING

Do not confuse the two costs. The naive loop wastes *computation* (recomputed K/V). The fix, the cache, trades that for *memory*; that is topics 5–6.

MATERIAL REFERENCE

WB p.1 (K/V table); NB1 sections "Why naive decoding is expensive", "Hand-trace", "Timing blow-up", Mermaid sequence diagram, Recap.

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB1 states that with a cache each step only does O(1) new work, so the total becomes O(n) instead of $O(n^2)$.
- NB1 builds a *CountingAttention* class that records how many tokens are processed per call, producing the 4,5,6,7,8 table above.
- NB1 setup: vocab 50, d_model 32; prompt 4 + 8 new tokens gives a (1, 12) output.

SAID IN CLASS, NOT IN MATERIALS

- The tutor's own 1,000-token example (999, 998, ... repeats).

Q&A

- **Q (Octavia Leland):** Why are we recalculating at all? **A:** That is how the architecture was defined in 2017; the cache came later.
- **Q (Divankar):** How is it quadratic? **A:** Because each step recomputes everything so far; the total adds up (n(n+1)/2).
- **Q (student):** Is that "exponential or logarithmic"? **A:** The tutor's reply was unclear (see Unresolved points); the formula shows it is quadratic.

3. The KV cache idea: what it is, where it lives, history

Save the K and V of tokens already processed and only compute K and V for the new token. The saved memory is the KV cache.

Key concepts

KEY CONCEPT

Simple: "Whatever you already calculated, save it somewhere, and only calculate the new thing".

Precise: a store of the Key and Value vectors for every token already processed, for every layer. At each decode step only the new token's K and V are computed and appended; the new Query attends over the cached K and V.

Explanation

- **Only K and V are cached, not Q and not outputs.** One new Query is needed per step; it is used and discarded. But that Query must attend over every previous token's Key and Value, and those never change (NB2). The tutor: "here you don't have to calculate any Q"; the memory formula is for K and V only.
- **Where it sits.** On the GPU, in VRAM, after the model weights are loaded. "If the model is not there, how will you calculate K and V?" [01:18:50].
- **History.** The 2017 "Attention Is All You Need" authors did save K and V manually but never used the term "KV cache"; the term became famous about three years later [00:42:27].
- **Inference only.** During training K and V are computed in parallel but nothing is saved; during inference they are saved [01:48:26].
- **Where it applies.** Wherever K and V are calculated: decoders (all major LLMs of the past 3–4 years are decoder-only), and also encoders and encoder-decoder (sequence-to-sequence) models, where the encoder's K and V passed through cross-attention are what gets cached [03:08:01]–[03:10:32].
- **On by default.** In Hugging Face Transformers *use_cache=True* is the default; you must disable it manually. Frameworks such as vLLM also do it by default when you self-host [03:03:22].
- **Same idea, different prefill/decode behaviour:** after prefill the cache is "ready" for decoding (topic 4).

EXAMPLE

At t3 of the whiteboard table the cache already holds K/V for <bos>, The. Only $K_{capital}$ and $V_{capital}$ are computed and appended. Nothing old is recomputed.

TIP

The tutor's summary of the fix (NB1): "cache K and V for tokens already processed; compute K/V only for the *new* token each step". But caching costs memory, which is the next problem.

MATERIAL REFERENCE

NB1 Recap; NB2 "Why only K and V get cached"; NB3 section "Prefill vs incremental decode" (past_kv); WB p.1.

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB3 shows the code-level view: in prefill, *past_kv=None*; in incremental decode the cached K/V is passed as *past_kv* and concatenated with the new token's K/V.

- The MQA paper (Paper §2.4) shows the same idea as *prev_K*/*prev_V* concatenated with the new K/V at each step.

SAID IN CLASS, NOT IN MATERIALS

- The history of the term "KV cache" (2017 → ~2020).
- The cache is never moved back to the CPU once on the GPU; K/V are first moved from RAM to VRAM [see topic 5 and Unresolved points].
- Practical cache lifetime and the difference between KV cache and prompt cache: see topic 7.

4. LLM inference: prefill and decode, TTFT and TPOT

Every request has two phases: read the whole prompt in parallel (prefill), then write the answer one token at a time (decode).

Key concepts

KEY CONCEPT

Inference = prediction with a trained model.

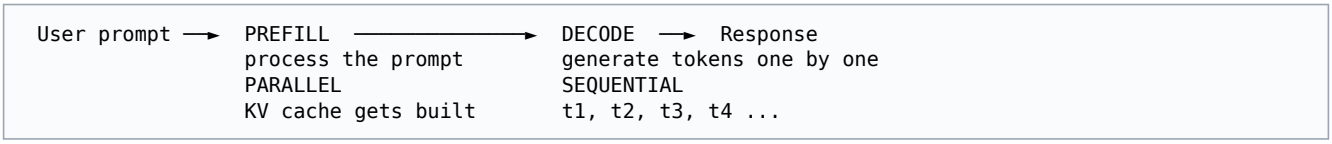
Prefill: process the whole user prompt; K and V for all prompt tokens are calculated in parallel and saved. When prefill finishes, the KV cache is ready.

Decode: generate tokens one at a time (sequential, by the autoregressive property). This is the slow bottleneck phase.

TTFT (time to first token): how long until the first token appears. Strongly affected by prefill.

TPOT (time per output token): time between successive output tokens. Associated with decode. Also used: **TPS** (tokens per second).

Explanation



Redrawn from WB p.2.

- **Prefill example (WB p.2).** "The capital of France" becomes 4 token IDs (101, 205, 401, 301); each becomes an embedding row (e.g. 0.48 ...); all tokens are processed together.
- **"Parallel" is not literal for huge prompts.** 1,000 prompt tokens may run as 500 in parallel and then another 500 (WB p.2).
- **Reading analogy.** Prefill is reading the whole newspaper; decode is writing a summary of it.
- **Why TTFT matters most.** A slow TTFT can mean waiting minutes, and nobody likes that. Typical TPOT averages 20–30 ms per token. To hide the long decode, applications use **streaming**: show each token as it is produced [01:03:12].
- **TTFT depends on prefill; TPOT on decode.** TPOT is computed from timestamps: time of token 2 minus time of token 1, etc. It can vary token to token (e.g. 20 ms, 21 ms).

The two example requests (WB p.3)

	Request A	Request B
Prompt	10,000 tokens	20 tokens
Output	20 tokens	10,000 tokens
Prefill	Expensive	Cheap
TTFT	Higher	Low
Decode	Short	Expensive (10,000 sequential tokens)

TIP

Inference engineering means handling both cases: long-prompt/short-answer (prefill-heavy) and short-prompt/long-answer (decode-heavy).

MATERIAL REFERENCE

WB p.2 (prefill → decode, parallel vs sequential, 101/205/401/301 example), WB p.3 (TTFT, TPOT, Requests A/B, "Streaming"); NB3 ("Prefill vs incremental decode").

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB3 prefill output: input (1, 5, 32) → output (1, 5, 32); cached K and V shape (1, 4, 5, 8) = (batch, num_heads, seq_len, head_dim).
- NB3 incremental decode: each step feeds a (1, 1, 32) token; cached K seq_len grows 6, 7, 8, 9, 10 over five steps.
- Paper §2.4: the incremental (decode-time) layer cannot be parallelised, because the output at one position decides the input at the next.

SAID IN CLASS, NOT IN MATERIALS

- TTFT, TPOT, TPS definitions, the 5–6 second prefill illustration, the 20–30 ms TPOT average, and the newspaper analogy.

Q&A

- **Q (Karthik):** Why is decode expensive? **A:** More tokens, produced one after another.
- **Q:** Does TPOT stay the same across requests A and B? **A:** No, it can vary; the first token might take 20–30 ms, the second 21 ms. TTFT is the one that matters more.
- **Q (Rushi):** Is there TTFT for image generation? **A:** That is not token generation, so it is different.
- **Q (Sridhar):** For summarisation (10,000-token input, ~300-token output) is something needed upfront? **A:** TTFT will be higher because more tokens must be processed.
- **Q (Viraj):** Can the cache be compressed? **A:** Yes: quantisation can be applied to the KV cache (as a later step, see topic 6).

5. The memory wall: compute throughput vs memory bandwidth

A GPU can compute far faster than memory can feed it data. During decode the GPU often waits for data, and that is the memory wall.

Key concepts

KEY CONCEPT

FLOP: one floating-point operation per second (the tutor's wording). **TFLOPS** = tera (trillion) floating-point operations per second. 1 TFLOPS = 1 trillion FP operations/s [01:05:59].

Compute throughput: how many operations the GPU can do per second. **Memory bandwidth:** how much data can be moved to the GPU per second.

Arithmetic intensity: how much computation you perform for every byte of data you move from memory [01:09:57].

Memory wall: the GPU is ready to compute but waits for data, because memory bandwidth cannot keep up.

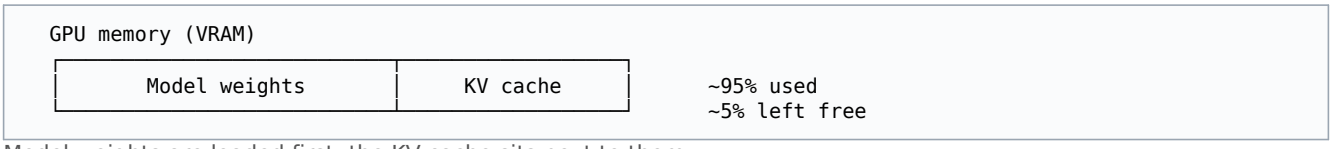
Explanation

- GPUs are rated in TFLOPS. Example numbers used on the board (WB p.4): **GPU throughput = 1000 TFLOPS**, **memory bandwidth = 2 GB/sec**. The tutor treated these as illustrative numbers, not a specific GPU.
- Data is not in the GPU by default; at every step it must be moved from RAM to the GPU's VRAM.
- **Case 1 (low arithmetic intensity):** load 1 GB of data and perform only 10 billion operations. Compared with 1,000 TFLOPS of capacity, that is very little computation per byte.
- **Case 2 (high arithmetic intensity):** load 1 GB and perform far more operations (WB p.5 says 1000 billion; spoken as "100 billion", see Conflicts).
- If memory bandwidth is low and compute is high, there is a bottleneck: the GPU waits. If nothing has been moved to the GPU there are no operations; the GPU simply waits for data.
- The goal is **high arithmetic intensity**: use as much of the GPU's TFLOPS as possible (the tutor said "try to process up to 95% of the limit", then added that 95 is only a rough figure).

EXAMPLE

Chef analogy (WB p.5). The chef can cook 100 dishes/s but the kitchen can only deliver ingredients for 20 dishes/s. The chef is idle most of the time; the bottleneck is the supply, not the cooking. That is the memory wall.

GPU memory layout (WB p.6)



Model weights are loaded first; the KV cache sits next to them.

- Model weights load first, because without weights you cannot compute K and V. The tutor said weights often take more than 40% of the memory (it depends on the model).
- Both weights and KV cache share VRAM. About 5% is kept free so that another process cannot crash the run.

- Both compute throughput and memory bandwidth should be high. High compute with low bandwidth wastes the GPU ("GPU waits").
- The problem is set by the **hardware** (processor, RAM generation e.g. DDR5, VRAM), not by what code you write. Faster processors out-run the memory system.
- Memory wall problem is felt most in the **decode phase** where one token is generated at a time. A smaller KV cache means less data to move, so better bandwidth use (this motivates MQA, topic 9).

WARNING

The tutor said the "95%" figure is not exactly correct and the 2 GB/s bandwidth is only an illustrative number. Do not quote either as a real hardware spec.

MATERIAL REFERENCE

WB p.4 (Memory Wall, TFLOPS, arithmetic intensity, Case 1), WB p.5 (Case 2, chef/kitchen, "Memory Wall – 95% – High Arithmetic Intensity"), WB p.6 (GPU box, RAM/SRAM, VRAM/GPU).

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- Paper §2.3.1–2.4.1 gives a formal version: batched MHA has memory-access/arithmetic ratio $O(1/k + 1/(bn))$; incremental decoding has $\Theta(n/d + 1/b)$, close to 1 when $n \approx d$ or $b \approx 1$, "causing memory bandwidth to be a major performance bottleneck". The paper notes GPU/TPU compute can be two orders of magnitude higher than memory bandwidth.

SAID IN CLASS, NOT IN MATERIALS

- Chef/kitchen analogy, the RAM vs VRAM transfer story, the 95% rule of thumb, and the hardware (Threadripper, DDR5) remarks. The notebooks do not cover the memory wall.
- Memory wall was described as "not part of your syllabus" but included because it relates to KV cache.

Q&A

- **Q (Arun):** How to explain the context window? **A:** It is simply the maximum sequence length of a model.
- **Q (Gaurav):** If K and V are computed independently, shouldn't everything be parallel? **A:** Independence is across tokens: for every token K and V can be computed in parallel (prefill).
- **Q (Kumal):** Difference between RAM and VRAM? **A:** VRAM belongs to the GPU; RAM is the normal system memory.
- **Q (Shivam):** Does context length depend on KV cache? **A:** Yes, the maximum number of tokens depends on it, and that creates another problem.
- **Q (Ayush):** How to architect the storage/fetching hardware for KV blocks? **A:** Use better hardware; software cannot fix this.
- **Q (MP):** How does the embedding stay fixed? **A:** The embedding dimension is chosen and not changed.
- **Q (Murtaza):** If arithmetic intensity is low, is the memory-bandwidth problem higher? **A:** Yes, and the GPU waits longer.

6. KV cache memory math

How many bytes does the cache need? A simple product of a few numbers; the tutor stressed this is the calculation to remember.

Key concepts

KEY CONCEPT

KV per token = 2 × num_kv_heads × head_dim × num_layers × dtype_bytes

KV cache bytes = KV per token × seq_len × batch_size

The 2 is for K **and** V. "Layers" means the number of transformer blocks (MHA 1, MHA 2, ...). dtype_bytes: FP16/BF16 = 2, FP32 = 4, FP64 = 8.

Worked example from the whiteboard (WB p.6–7)

Assumptions: KV heads = 8; head dimension = 64 (512 / 8 = 64, from the original paper's d_{model} = 512 and 8 heads); layers = 32; FP16 = 2 bytes.

Step	Calculation	Result
1. K per layer	8 heads × 64 dimensions	512
2. V per layer	8 heads × 64 dimensions	512
3. K + V together	512 + 512	1,024 numbers
4. Bytes per token per layer	1,024 × 2 bytes (FP16)	2,048 bytes = 2 KB
5. All 32 layers	2 KB × 32	64 KB per token
6. 10,000 tokens	10,000 × 64 KB	640 MB
7. 100,000 tokens	100,000 × 64 KB	6.4 GB

CONFLICT

Transcript: while speaking the tutor said "let's take 100K ... 640 MB". **Whiteboard (WB p.7):** 10,000 tokens = 640 MB, and 100,000 tokens = 6.4 GB. The board arithmetic is consistent (64 KB × 100,000 = 6.4 GB), so the spoken line looks like a slip. Both versions are shown here; the whiteboard figures are used in these notes.

What changes the number

- **Linear growth.** Cache size grows linearly with sequence length and with batch size (and multiplicatively when both grow) (NB2).
- **Data type.** FP16 → FP32 doubles the cache. The tutor said most LLM work uses 16-bit; 32 is "not a preferred choice". FP64 would be 8 bytes.
- **Layers.** More parameters usually means more layers, so a bigger cache: more layers → more parameters → more KV cache memory.
- **Batch size.** Default 1 for one request. If you run 10 or 16 sequences in parallel, the cache grows accordingly.
- **Inference only.** This is entirely an inference-time cost (training does not save K/V).

Which term can we reduce?

Term	Can it be changed?	Tutor's comment
dtype_bytes	Yes, via quantisation	Possible but "our last solution", not the first step

Term	Can it be changed?	Tutor's comment
num_layers	No	Fixed by the model
head_dim	No (not the target)	Cannot change
num_kv_heads	Yes, and this is the lever	In MHA num_kv_heads = number of heads. The next algorithms change it (MQA, GQA).

WARNING

Q does not appear in the memory formula. When a student asked "why not Q?", the answer was: this is the K and V cache math; Q is not stored.

Numbers from the notebooks

Model (NB2, seq_len 4096, FP16, batch 1, full MHA)	Layers	Heads (=KV heads)	Head dim	Cache
7B-ish	32	32	128	2.00 GB
13B-ish	40	40	128	3.12 GB
70B-ish	80	64	128	10.00 GB

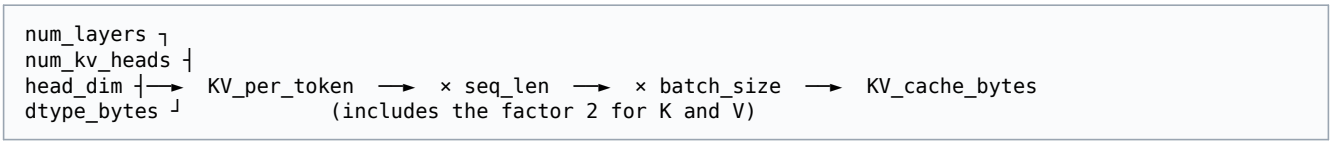
7B-ish, 4096 tokens	Value
FP16/BF16 (2 bytes)	2.00 GB
FP32 (4 bytes)	4.00 GB
Batch 1 / 4 / 16 / 64 (FP16)	2 GB / 8 GB / 32 GB / 128 GB

KEY CONCEPT

NB2 takeaway: cache bytes scale linearly with seq_len and with batch_size, but together they multiply. That is why serving many long-context requests at once is "a memory problem, not just a compute problem".

MATERIAL REFERENCE

WB p.6–7 (worked example); NB2 formulas, three tables, "Try it yourself", Mermaid flowchart; flowchart redrawn below.



From the NB2 flowchart.

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB2 "Try it yourself": 32 layers, 8 KV heads, head dim 128, seq_len 8192 (a "GQA-8" config) gives 128 KB per token and 1.00 GB total.
- NB2 recap: every attention variant in the course pulls the *num_kv_heads* lever: MHA uses all, MQA uses 1, GQA something in between, MLA replaces heads with a compressed latent dimension.
- NB2 refers to a file *reference_tables/model_config_memory_worksheet.md*, which was not provided.

EXTRA CONTEXT (NOT FROM CLASS)

- The notebook's "GB" values are binary (2.00 GB = 2^{31} bytes ≈ 2.15×10^9), so they are slightly different from decimal GB. The whiteboard's "640 MB" is a rounded decimal figure.

- Derived from the whiteboard numbers (not stated in class): at FP32 the same example is 128 KB per token, i.e. 1.28 GB for 10,000 tokens and 12.8 GB for 100,000 tokens. This matches the "12.8 GB" figure the tutor started to say, though the transcript is unclear.

SAID IN CLASS, NOT IN MATERIALS

- The step-by-step FP16 walk-through with 8 KV heads, 64 dims, 32 layers; the 10,000 / 100,000 token examples; FP64 = 8 bytes; "quantisation is the last step".

Q&A

- **Q (Rahul):** Where does Q appear in the KV memory math? **A:** It does not; only K and V are cached.
- **Q (Ramdas):** Put light on the number of KV heads. **A:** Each head has its own K and V in MHA; that is what we have discussed since the transformer architecture.
- **Q:** Is this a training or an inference view? **A (tutor, to the class):** inference; in training K/V are computed in parallel but not saved.

7. KV cache in practice: lifetime, eviction, prompt cache

How a real serving system decides how long to keep the cache. Most of this came from student questions and is the tutor's practical advice, not slide or notebook content.

Explanation (as the tutor described it)

- **One cache per request/user.** A different prompt has its own K/V, so the cache of one user is of no use to another. Keeping a cache for every user in GPU memory is not practical, so memory must be freed over time [02:28]–[02:38].
- **When to clear.** It depends on VRAM and on the length of prompts. With plenty of VRAM you can keep the cache for hundreds of requests (the tutor gave "after 200 or 300 requests" and "500 / 1,000" as examples). With little VRAM, clear after every request or after a few. Prompts of unusual length (e.g. 15,000 tokens) can force earlier clearing. Sequential and parallel requests both accumulate cache.
- **Budget example.** On an 80 GB H100 with a 20 GB model, roughly 50 GB could be given to the KV cache. A split such as 20–30% model and 60–70% KV cache was mentioned [02:38:12]. More cache memory means more requests served before clearing.
- **Stays on the GPU.** The KV cache must live in GPU memory to be useful; it is not moved back to CPU RAM, and is cleared when no longer needed (the tutor rejected "store it in RAM").
- **Not a normal cache.** It is not like Redis. Conversation history for each user is kept separately (e.g. in Redis) and has nothing to do with the KV cache [03:12:32].
- **TTL.** Asked whether the cache expires after a TTL, the tutor said it is not simply that: it depends on memory. If memory is short the cache is removed for the next inference; if there is memory it can stay.
- **Context window is a setting.** The same model can be offered with 1 M tokens by one provider and a smaller window by another. If you control the weights/serving, you can change it; with a hosted API like ChatGPT you can only limit generation tokens [03:13:14].
- **Prompt cache vs KV cache.** Providers (e.g. on OpenRouter) show "cached prompt tokens". The tutor said these are two different caches: prompt cache usually expires very fast (about 5–10 minutes), KV cache can live a bit longer; details differ by provider. Anthropic has a blog on prompt caching.
- **Prefix caching** (reusing the pre-fill of a shared prefix, e.g. one organisation-wide system prompt) will be covered later with paged attention.

TIP

Rule of thumb from the tutor: look at input tokens and available VRAM, then write your own algorithm for when to clear. Less VRAM → clear sooner.

MATERIAL REFERENCE

NB2 batch-size table (only the batch scaling is in the materials). The rest is class discussion.

SAID IN CLASS, NOT IN MATERIALS

Everything on this page except the batch-size effect: clearing policy, H100 example, Redis vs KV cache, prompt cache vs KV cache, context-window setting.

Q&A

- **Q (Murtaza):** Is the cache cleared after a TTL? **A:** Not exactly; if memory is low it is removed for the next request, if memory is available it may stay.
- **Q (Prasanjay):** The cache fills quickly; how is it deleted? **A:** Handled by the inference engine; configurable (after 4, 10, 100 requests, depending on VRAM).

- **Q (Ramesh):** How to decide when to evict/offload users? **A:** By VRAM and the input sequence length; reserve a share of VRAM for KV cache.
- **Q (Saravana):** Can the KV cache be stored in RAM? **A:** No, it should be on the GPU.
- **Q (Prasanna, live):** Is the "cached prompt tokens" shown by providers the KV cache or a prefix cache? **A:** There are two kinds: KV cache and prompt cache; it depends on the provider; prompt cache expires in about 5–10 minutes; prefix caching comes later.

8. Multi-Head Attention (MHA) recap

The baseline. Each head has its own Q, K and V; heads are concatenated. In MHA, num_kv_heads equals the number of heads, so the cache is the largest.

Key concepts

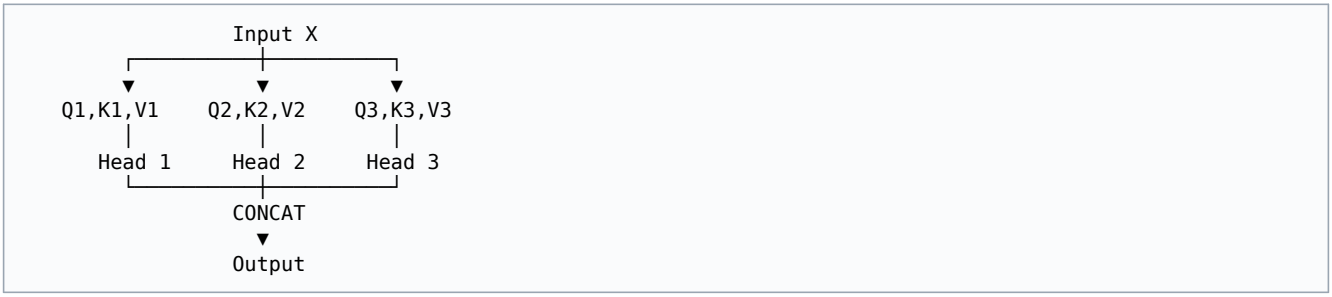
KEY CONCEPT

Simple: several attention "viewpoints" run in parallel, then their results are joined.

Precise: d_model is split into num_heads heads of size head_dim = d_model / num_heads. Each head computes scaled dot-product attention with its own Q, K, V; the head outputs are concatenated and mixed by an output projection (NB3).

Different heads can track different relationships (NB3 example: one head for syntax, another for coreference).

Explanation



Redrawn from WB p.8 (3 heads drawn; the tutor then used 8 heads: H1 → Q1,K1,V1 … H8 → Q8,K8,V8).

- For 8 heads: 8 Q, 8 K and 8 V per token, i.e. 24 in total (tutor's count: "3 per head, 8 heads").
- Using d_model = 512 and 8 heads, each head has dimension 64 (WB p.11).
- The cache then stores K and V for **every head**, which makes decode inference expensive. The idea to fix this (WB p.8 last line): "Idea → unified KV".
- Each head learns its own matrices W_Q, W_K, W_V; the tutor reminded the class that this is what was covered in the annotated-transformer session.

MATERIAL REFERENCE

WB p.8; NB3 (MHA recap, sections "Why multiple heads", "Per-layer cache growth", flowcharts); Paper §2.2–2.4 (code for multi-head attention).

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB3 sample: d_model = 32, num_heads = 4, head_dim = 8. With a 3-token prompt and 4 layers, every layer's cache has length 3 after prefill and 5 after two decode steps.
- NB3 state diagram: Prefill → Decode (past_kv = (K,V) from the prompt); Decode → Decode (past_kv grows by 1 token).
- NB3 recap: "MHA caches num_heads full K/V heads per layer, the most expensive point on the memory-vs-quality spectrum".
- Paper §2.2: MHA uses learned projections P_q, P_k, P_v with shapes [h, d, k]/[h, d, v] and P_o for the output; the scaling factor on logits is omitted in the code because it can be folded into the projections.

Q&A

- **Class check (tutor to participants):** with 8 heads, how many Q/K/V in total? **Answer reached:** 3 × 8 = 24.
- **Q (Ashok):** Do matrix operations change if projections change? **A:** Yes, they change when the projection shapes change (see MQA).

9. Multi-Query Attention (MQA)

Keep all the query heads, but let every head share one K and one V. The cache shrinks by the number of heads, at a small cost in attention diversity.

Key concepts

KEY CONCEPT

Simple: many questioners (queries), one shared answer sheet (K and V).

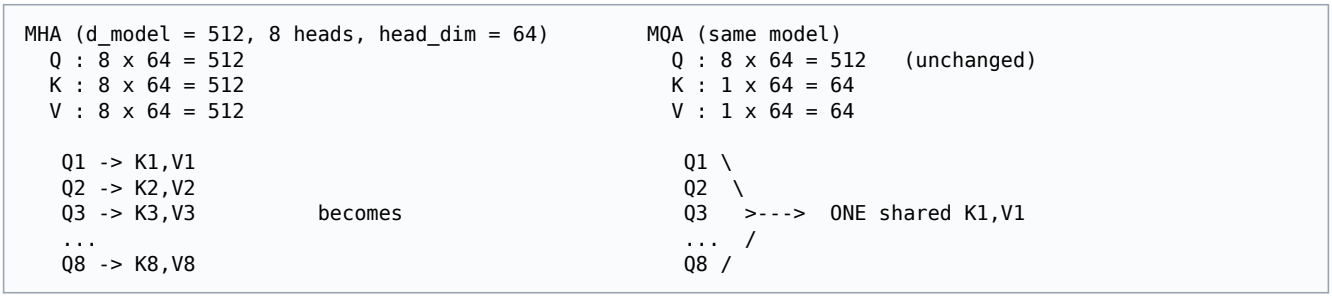
Precise: all num_heads query heads attend against a single shared Key head and a single shared Value head, so the K/V part of the cache falls by a factor of num_heads (NB4). The paper: "the different heads share a single set of keys and values".

Do not confuse: the number of heads does **not** change; the number of **K/V heads** (and K/V projection dimension) does.

MHA vs MQA at a glance (WB p.9)

	MHA	MQA
Query	Multiple	Multiple
Key	Multiple	One
Value	Multiple	One
KV cache	Large	Small
Attention diversity	Higher	Lower
Q projection	512	512 (unchanged)
K, V projection	512 each	64 each (dimension changes)

How the shapes change



Redrawn from WB p.10–12. Each query (64-wide) uses the same K1 and V1; concatenating the 8 query-head outputs still gives 512.

- **Change the projection, not the heads.** This was the tutor's most repeated point. The K and V projections output a smaller dimension (e.g. 64; 128 is also possible, but it must be lower than the embedding dimension). Q stays at 512. Q can in principle be made larger, never smaller.
- **32-head example (WB p.11).** 32 query heads and 1 KV: Q = 32, KV = 1, so the KV cache is 32 times smaller (a student's "32× less" was confirmed).
- **The attention formula does not change:** $\text{softmax}(QK^T/\sqrt{d_k})V$ (WB p.11).
- **No dimension mismatch** because only K and V are projected smaller; every query head reads the same 64-wide K and V.
- **Different heads still behave differently.** Sharing K and V does not make every head produce the same attention scores, because each head still has its own Q. (WB p.10: "never produces same

patterns every head".)

- **K and V are generated once per token** and then read by all query heads. The heads do not update the shared K/V independently (see Q&A).
- **Training:** no change in procedure; back-propagation is the same; the K and V matrices are simply smaller. The model must be **trained as MQA**; you cannot change an MHA-trained model to MQA afterwards [03:23:51].

Benefits and costs

KEY CONCEPT

Benefits: smaller KV cache (by num_heads×), less K/V data to move between memory levels (the tutor linked this to the memory wall), faster generation, more room for compute and other requests.

Cost: lower attention diversity ("context richness"); quality drops slightly. The tutor quoted roughly 1–2% as acceptable "according to the original literature" [02:07:20].

TIP

MQA is the maximum-compression extreme; GQA (next class) is the tunable middle ground between MHA (8 K/V heads) and MQA (1 K/V head). The tutor said that in 2026 he leans towards GQA rather than MQA.

WARNING

Do not say "MQA reduces the number of heads". It reduces the number of K/V heads (and their projection size) only. Queries and total head count stay the same.

MATERIAL REFERENCE

WB p.8–12; NB4 sections "Derivation", "The tradeoff", Mermaid diagram, "Try it yourself"; Paper §3 and §3.1.



NB4 Mermaid diagram.

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB4 shapes: MHA cached K = (1, 4, 5, 8) with "num_heads distinct K heads"; MQA cached K = (1, 1, 5, 8), "always exactly 1 K head".
- NB4 cache table (4096 tokens, FP16):

Model	MHA cache	MQA cache	Reduction
7B-ish (32 layers, 32 heads, hd 128)	2.00 GB	64.00 MB	32x
13B-ish (40 layers, 40 heads, hd 128)	3.12 GB	80.00 MB	40x
70B-ish (80 layers, 64 heads, hd 128)	10.00 GB	160.00 MB	64x

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- NB4 trade-off text: fewer distinct K/V subspaces means less representational capacity for attention patterns; this is "exactly why GQA exists as a middle ground". The MQA cache size never changes if you change num_heads; only MHA's does.
- The NB4 shapes show one K head of head_dim size, matching the whiteboard's "1 × 64 = 64" (a single head-sized K and V).

- Paper §3 code: "identical to the code for multi-head attention, except that we remove the letter h from the tf.einsum equations" for K, V, P_k, P_v. Paper §3.1: the offending n/d term of the memory-to-compute ratio is reduced by a factor of h (ratio $\Theta(1/d + n/(dh) + 1/b)$).

SAID IN CLASS, NOT IN MATERIALS

- Quality drop of about 1–2%; GQA arrived around 2023 and is more used now; DeepSeek's MLA; "in 2026 I lean to GQA".
- MQA (2019) "nobody is using it" now; the idea of one shared head 1..H4 → one KV pair.
- The CPU → GPU transfer remark (WB p.10, "K,V → CPU → GPU") and "smaller memory is easier to transfer".
- Hallucination is not the right term for reduced attention diversity.

Q&A

- **Q (Ashwini):** If you reduce heads, is context richness reduced? **A:** Yes, that is the con (attention diversity is lower).
- **Q (Paritosh):** Is the effect "hallucination"? **A:** No; hallucination is not the correct term.
- **Q (Murtaza):** How will different heads store their K and V? **A:** Save them per token (e.g. a dict); but with MQA only one shared K/V is stored.
- **Q (Abhishek):** How are W_K and W_V trained across heads? **A:** The matrices just have a changed shape; back-propagation works the same, updating the (smaller) K and V values.
- **Q (Gaurav):** Can Q also be reduced? **A:** No; Q must be kept (512). It can be higher, not lower.
- **Q (Niha):** How is resizing done by changing the projection? **A:** A dimension change of the K and V projection matrices.
- **Q (Alok):** MQA has lower dimension, GQA balanced? **A:** Yes, that is tomorrow's topic.
- **Q (Acharya):** Memory is reduced but information richness is lower? **A:** Exactly.
- **Q (Ramesh):** Is the 2019 paper still valid? **A:** Yes; but in 2026 he is more aligned with GQA.
- **Q:** How much quality is lost? **A:** Judged from the final response quality (see topic 11 for the paper's numbers).

10. Demo: generation with and without the KV cache

The Colab notebook times the same 300-token generation with use_cache=False and use_cache=True.

Setup

- Model: **HuggingFaceTB/SmolLM2-1.7B** (a small 1.7 B model), loaded in float16 on CUDA (float32 on CPU). Use at least a **T4 GPU** in Colab.
- Prompt: **"The red cat was"** (4 input tokens); generate **300 new tokens** greedily (*do_sample=False*).
- Helper *measure_generation_time(use_cache, runs=3, max_new_tokens=300)*: one warm-up run, then 3 timed runs with *torch.cuda.synchronize()* before and after; returns the average. Uses *torch.no_grad()* (prediction only) and *model.eval()* (inference mode).
- In Transformers **use_cache=True is the default**; the tutor disabled it manually for the first test.

Results

Quantity	Said / seen in class	Saved in the uploaded notebook
Average time, no cache	≈ 15.77 s ("15.773")	17.617 s
Average time, with cache	≈ 8 s	8.171 s
Time saved	≈ 7 s	9.446 s
Reduction	48.8%	53.62%
Speed-up	≈ 1.9× (spoken as "1.9 minutes")	2.16×

CONFLICT

The class run (about 15.8 s vs about 8 s, 48.8%, 1.9×) and the saved notebook outputs (17.6 s vs 8.2 s, 53.6%, 2.16×) differ. They are separate runs of the same code on different sessions, so both sets are shown. Either way the cache roughly halves the time.

KEY CONCEPT

In the saved outputs both runs begin with identical text (greedy decoding), so the cache changes speed, not the answer. The longer the generation, the bigger the saving, because the no-cache cost grows quadratically (topic 2).

MATERIAL REFERENCE

Copy_of_KVCACHE_DEMO.ipynb, cells 0–4.

Q&A

- **Q (Sridhar, live):** Why did you set use_cache true/false? **A:** "Just to showcase": by default it is on; the false run disables it to show the time difference.

11. The MQA paper: "Fast Transformer Decoding: One Write-Head is All You Need"

Noam Shazeer (Google), arXiv 1911.02150v1, 6/7 November 2019. The tutor walked through the abstract, the code, and the results tables.

Key ideas

- **Problem.** Training attention layers is fast (parallel over the sequence), but incremental inference is slow because of the memory bandwidth cost of repeatedly loading the large K and V tensors (Abstract; Introduction).
- **Idea.** Multi-query attention: keys and values are shared across all attention heads. Quality drops only slightly.
- **Analysis (Paper §2.3.1, §2.4.1, §3.1).** With assumptions m = n, k = v = d/h, n ≤ d: arithmetic operations $\Theta(bnd^2)$ in all cases. Memory-to-compute ratio: batched MHA $O(1/k + 1/(bn))$; incremental MHA $\Theta(n/d + 1/b)$; incremental MQA $\Theta(1/d + n/(dh) + 1/b)$. The offending n/d term is cut by a factor of h. A larger batch b removes the 1/b term.
- **Experiments (§4).** WMT 2014 English–German translation (encoder–decoder Transformer, 6 layers, d_{model} 1024, d_{ff} 4096, h = 8, $d_k = d_v = 128$, 211 M parameters, 100,000 steps, 32-core TPUv3) and Billion-Word language modelling (decoder-only, 192 M parameters). In the MQA model d_{ff} is widened (4096 → 5440 for translation) so parameter counts match.

Table 1: WMT14 EN–DE quality (paper)

Attention	h	d_k,d_v	d_{ff}	ln(PPL) dev	BLEU dev	BLEU test beam 1 / 4
multi-head	8	128	4096	1.424	26.7	27.7 / 28.4
multi-query	8	128	5440	1.439	26.5	27.5 / **28.5**
multi-head local	8	128	4096	1.427	26.6	27.5 / 28.3
multi-query local	8	128	5440	1.437	26.5	27.6 / 28.2
multi-head	1	128	6784	1.518	25.8	
multi-head	2	64	6784	1.480	26.2	26.8 / 27.9
multi-head	4	32	6784	1.488	26.1	
multi-head	8	16	6784	1.513	25.8	

Table 2: amortised cost, µs per output token on TPUv2 (paper)

Attention	Training	Inference enc. + dec.	Beam-4 enc. + dec.
multi-head	13.2	1.7 + 46	2.0 + 203
multi-query	13.0	1.5 + 3.8	1.6 + 32
multi-head local	13.2	1.7 + 23	1.9 + 47
multi-query local	13.0	1.5 + 3.3	1.6 + 16

Table 3: Billion-Word LM, dev perplexity (paper)

Attention	h	d_k,d_v	d_{ff}	dev-PPL
multi-head	8	128	8192	29.9
multi-query	8	128	9088	30.2
multi-head	1	128	9984	31.2

Attention	h	d_k,d_v	d_{ff}	dev-PPL
multi-head	2	64	9984	31.1
multi-head	4	32	9984	31.0
multi-head	8	16	9984	30.9

What the tutor emphasised

- In BLEU the multi-query model (26.5) is only slightly below multi-head (26.7). The simpler ways of shrinking K/V (fewer heads or smaller d_k, d_v) hurt much more (25.8–26.2).
- MQA with beam-4 decoding had the highest test BLEU (28.5) [02:47:39]. Beam search (deterministic) was explained as looking ahead beyond the next token; top-k/top-p are stochastic. Beam search is old and not relevant for newer LLMs.
- The real win is on the inference side: decoder time per token is 46 µs for MHA vs 3.8 µs for MQA (paper: 47 ms vs 3.9 ms per decoding step). So MQA generates faster and TTFT improves, while quality is "very, very close".
- The tutor stressed not to think of MQA as a game-changer for core quality; the gain is the KV cache. This is why a middle ground (GQA) was developed, which is tomorrow's topic.
- The paper was not in a conference; it was posted on arXiv.

CONFLICT

In class the Billion-Word multi-query value was read as "32.2". Table 3 of the paper says **30.2** (29.9 for multi-head). The paper value is used here.

MATERIAL REFERENCE

1911_02150v1.pdf pp. 1–9; WB has no paper content.

ADDITIONAL INFO FROM MATERIALS (NOT SAID IN CLASS)

- Training speed is almost identical (13.2 vs 13.0 µs). Local attention plus MQA gave the lowest decode cost (3.3 µs; 16 µs in beam-4).
- Paper §4.2 evaluates the dev set with greedy decoding and the test set with beam 4, $\alpha = 0.6$.
- The paper says the experiment used padded fixed-shape memory tensors (footnote 1), so each decode step took the same time.

Q&A

- **Q:** How much quality is lost with MQA? **A:** Tables 1 and 3: BLEU 26.7 → 26.5; perplexity 29.9 → 30.2; much smaller loss than reducing heads or head size.

12. Live Q&A at the end of class [02:58]–[03:24]

Questions asked by participants speaking on the call, with the tutor's answers.

Q&A

- **Murtuza Saifee [02:58]:** In MQA all heads share K/V; will they update the same values at the same time, and will it conflict? **A:** K and V are generated **once per token** and then shared and read by all query heads. Heads do not update them independently; only one is generated. For training the process is unchanged. Q values are generated per head, not K and V.
- **Sridhar K [03:02]:** If input is short and the output is long (e.g. build a website), does the KV cache help? **A:** Yes; it is on by default (Transformers, vLLM); you never enable it manually. The false/true setting in the demo was only for showing the difference.

- **Sridhar K:** Can a KV cache for a fixed prompt be shared across a team? **A:** In principle, but the GPU infrastructure cost would be very high because each user's cache has to be held; usually it is released after N requests depending on VRAM and the input length.
- **Gaurav Garg [03:06]:** Why not reduce Q size too? **A:** Keep Q at 512; you cannot lower it. K and V are reused 8 times; we keep the Q dimensions the same, not the information (each token's Q still differs).
- **Gaurav:** Which architectures use KV cache: encoder–decoder or decoder? **A:** Wherever K and V are calculated: the original 2017 Transformer too. Modern LLMs are decoder-only. In seq2seq models cross-attention carries the encoder K/V, which are saved.
- **Gaurav:** Do GPT-style models use cross-attention? **A (partial):** The tutor discussed T5-style sequence-to-sequence models; cross-attention existed even before 2017 (Bahdanau attention, 2014). Gaurav left saying he needed to read more.
- **Gaurav:** Does the cache get replaced when a different user comes? **A:** Not necessarily every time; clear after e.g. 200–300 requests, or after every request if VRAM is tight.
- **Gaurav:** How is the final prompt/response stored? **A:** A separate, ordinary cache (e.g. Redis); not a KV cache.
- **Gaurav:** Can max tokens be changed? **A:** For hosted models like GPT you can limit the max generation tokens, but not the context window; with your own weights you control it.
- **Paritosh Gupta [03:14]:** (hackathon idea) Build an HPC assistant that benchmarks configs (e.g. QWEN distillation) on the Leonardo supercomputer and gives cold-start configs/scripts or an MCP tool. **A:** Possible; start with one GPU, then 4, then 8, before scaling. Try the teacher/student model sizes first (e.g. a 27 B teacher and a ~2 B student), use existing calculators as a baseline, and consider multiple teachers. Scripts and an MCP tool are feasible. Once the hackathon is over, the cluster access ends. In 2026, scaling laws for reinforcement learning are a more current topic than the 2021–22 LLM scaling laws.
- **Prasanna Sivaneni [03:21]:** What is the "cached prompt tokens" shown by OpenRouter-style providers: KV cache or prefix cache? **A:** There are two kinds: KV cache and prompt cache. Prompt cache expires in minutes; KV cache can be a bit longer. Prefix caching will be discussed with paged attention.
- **Arun Subramanian [03:23]:** MQA is for generation, but the model must be trained with MQA instead of MHA, right? **A:** Correct. You cannot change an MHA-trained model into MQA afterwards.

SAID IN CLASS, NOT IN MATERIALS

Everything in this section came only from the call.

13. Logistics, roadmap and tangents

Short admin items and non-technical remarks (kept brief, as instructed).

- **Resources.** The repo link, Notion page (class links by date: e.g. 5th and 7th September class notes) and Discord link were shared in the chat; new joiners should check Notion and the syllabus before asking. The tutor asked students not to ask repeatedly for GitHub/Colab links.
- **Repo setup (optional).** Clone the repo (not required), create a virtual environment, install requirements. Running the notebooks is not compulsory; the aim is to follow the calculations. There are 4–5 notebooks pushed so far (and about 10 more to come); notebook 02 is the KV memory notebook.
- **Next class (tomorrow):** GQA, MLA, rest of the variants, possibly starting RoPE (positional encoding upgrade), and finally paged attention.
- **Tangents (not class content):** plans for a new advanced cohort in December (project/scaling, SLMs, agents + RAG), a mention of a launch "tomorrow at 8 AM" [unclear: what "FD" refers to], tool-use with Claude Code being non-deterministic, SSMs / Jamba in a future cohort. Nothing here is examinable.
- **Break.** Taken around [01:24] for 10 minutes.

SAID IN CLASS, NOT IN MATERIALS

All of this section.

Key takeaways

- 1. Decoding is autoregressive: one token at a time, each attending to all earlier tokens.
- 2. Without a cache, K and V of every old token are recomputed at every step: total work n(n+1)/2, quadratic in n.
- 3. The KV cache saves K and V (not Q, not outputs) for all processed tokens in GPU VRAM, so each step only computes the new token's K and V.
- 4. Inference has two phases: prefill (parallel, builds the cache, determines TTFT) and decode (sequential, determines TPOT).
- 5. The memory wall: GPU compute (TFLOPS) far exceeds memory bandwidth, so the GPU waits for data; the goal is high arithmetic intensity.
- 6. KV per token = 2 × KV heads × head_dim × layers × dtype bytes; total = × seq_len × batch. Example: 8 × 64 × 32 layers, FP16 = 64 KB per token (640 MB for 10,000 tokens).
- 7. The cache grows linearly with sequence length and batch size; the main lever to shrink it is num_kv_heads (quantisation is the last resort).
- 8. MHA: every head has its own Q, K, V. MQA: all query heads share one K and V; the number of heads is unchanged, only the K/V projection shrinks (e.g. 512 → 64).
- 9. MQA cuts the KV cache by the number of heads (32× for 32 heads) at a small quality cost (paper: BLEU 26.7 → 26.5); the main benefit is faster decoding.
- 10. MQA must be built in at training time; GQA is the middle ground, and the next class covers GQA, MLA and paged attention.

Glossary

Term	Meaning
Token / token ID	A word piece and the integer assigned to it by a tokenizer.
Embedding	The vector looked up for a token ID.
Q, K, V	Query, Key, Value vectors each token produces for attention.
Scaled dot-product attention	$\text{softmax}(QK^T/\sqrt{d_k})V$.
Causal mask	Sets future-position scores to $-\infty$ so a token cannot see later tokens.
Autoregressive	Generating one token at a time, feeding each output back as input.
<bos> / <eos>	Beginning / end of sequence tokens.
Naive decoding	Decoding with no cache: all K/V recomputed every step.
KV cache	Stored K and V of all processed tokens, per layer, kept in GPU VRAM.
Inference	Prediction with a trained model.
Prefill	Phase that processes the whole prompt in parallel and builds the cache.
Decode	Phase that generates tokens one by one.
TTFT / TPOT / TPS	Time to first token / time per output token / tokens per second.
Streaming	Showing tokens as they are generated.
FLOP / TFLOPS	Floating-point operation(s) per second / trillions of them.
Memory bandwidth	How fast data can be moved to the GPU.
Arithmetic intensity	Computation done per byte moved from memory.
Memory wall	Compute outpaces memory bandwidth, so the GPU waits.
VRAM / RAM / SRAM	GPU memory / system memory / on-chip memory (as named by the tutor).

Term	Meaning
Context window	Maximum sequence length of a model.
MHA	Multi-Head Attention: separate Q, K, V per head.
MQA	Multi-Query Attention: all query heads share one K and V.
GQA / MLA	Grouped-Query / Multi-head Latent Attention (covered next class; only named today).
Attention diversity	How varied the attention patterns across heads can be (lower in MQA).
Quantisation	Storing numbers in lower precision (e.g. for the KV cache).
Beam search	Deterministic decoding that looks ahead over several candidate sequences.
Prompt cache	Provider-side cache of prompt processing, distinct from the KV cache; expires in minutes.

Action items, homework and deadlines

- No deadlines or graded assignments were stated.
- Optional setup: clone the repo, create a virtual environment, install requirements, run notebooks 00–04 (execution is optional).
- Open the Colab demo (use a T4 GPU) and compare use_cache=False/True.
- Review MHA and the annotated-transformer material beforehand; revise the formulas (attention, KV per token).
- Read the MQA paper (arXiv 1911.02150).
- Look up beam search if unfamiliar (tutor: a small topic).
- Check the Notion syllabus; join the Discord.
- For tomorrow: GQA, MLA, possibly RoPE, then paged attention.

Resources and tools mentioned

- Course repo, Notion page, Discord (links shared in chat only, not in the transcript).
- Colab notebook for the cache demo (link shared in chat).
- Paper: Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need" (arXiv:1911.02150).
- Anthropic blog on prompt caching (named by the tutor).
- Tools: PyTorch, Hugging Face Transformers, SmolLM2-1.7B, vLLM, Redis, Cursor (editor), Rich (tables), Google Colab.
- Earlier course material referenced: annotated transformer notebook, mask-creation notebook, last week's fine-tuning class.

Unresolved or unclear points

- "Exponential or logarithmic?" The tutor's answer was unclear; the correct statement from the formula is quadratic (and KV memory is linear).
- The tutor also said once that KV cache "grows exponentially"; the formula and notebook say linear in sequence length and batch size.
- Case 2 of arithmetic intensity: spoken "100 billion" vs board "1000 billion" ops.
- Where K and V are computed: the tutor mentioned CPU → GPU transfer (WB p.10) and also that the cache lives on the GPU; the spoken explanation was not fully clear.
- Why exactly "95%" is "not correct": he only said to leave ~5% free; no precise reason given. The "more than 40%" weights share was also approximate.

- Whether GPT-style (decoder-only) models use cross-attention: Gaurav's question was not fully resolved in the call.
- Exact way quality loss was measured for MQA: answered only as "based on the final response".
- Whether Q can be made larger than 512 (tutor: "can be increased, not lowered"): not demonstrated.

Quick revision cheat sheet

KEY CONCEPT

Naive decoding work = 1 + 2 + … + n = n(n+1)/2 (quadratic). **With cache:** only the new token's K,V each step.

KV per token = 2 × KV_heads × head_dim × layers × bytes. **Cache** = per token × seq_len × batch.

Worked: 8 × 64 × 32 layers × 2 (K,V) × 2 bytes = 64 KB/token → 10,000 tokens = 640 MB → 100,000 tokens = 6.4 GB.

Bytes: FP16/BF16 = 2, FP32 = 4, FP64 = 8. Only K and V cached (never Q). Inference only.

Prefill	Decode
Whole prompt, parallel; builds the cache	One token at a time, sequential
Drives TTFT (long prompts = slow)	Drives TPOT (~20–30 ms/token; long outputs = slow)

	MHA	MQA
Query heads	8	8 (same)
K / V heads	8 / 8	1 / 1
Q, K, V projection (d=512)	512, 512, 512	512, 64, 64
KV cache	Large	Smaller by number of heads (32 heads → 32×)
Attention diversity / quality	Higher	Lower; slight quality drop (BLEU 26.7 → 26.5)
Decode speed (paper)	46 µs/token	3.8 µs/token

TIP

Memory wall: GPU compute (TFLOPS) ≫ memory bandwidth → GPU waits (chef cooks 100 dishes/s, kitchen supplies 20). Aim for high arithmetic intensity.

Practice: KV cache lives in GPU VRAM next to the weights; clear after N requests depending on VRAM; prompt cache ≠ KV cache; use_cache=True is the default.

WARNING

MQA changes the **K/V projection and K/V head count**, not the number of query heads. It must be trained in, not swapped in after training. Quantising the cache is the last resort.